Stage 4: Offline serving. An inference engine loads the quantised ONNX model and a local SentencePiece tokenizer and decodes greedily. A router resolves each request to a directly trained model where one exists, and otherwise chains two students through English. One engine serves a REST API, a command-line tool, an offline progressive web application and a browser client.

2.6.1 The Deciding Experiment

The project's founding hypothesis was H1: *preference distillation produces better compact students than sequence-level knowledge distillation*. It was tested at matched student size, on identical data, with evaluation on real human references held out from training, a design that is fair regardless of what each system trained on. Figure 2.1 gives the outcome.

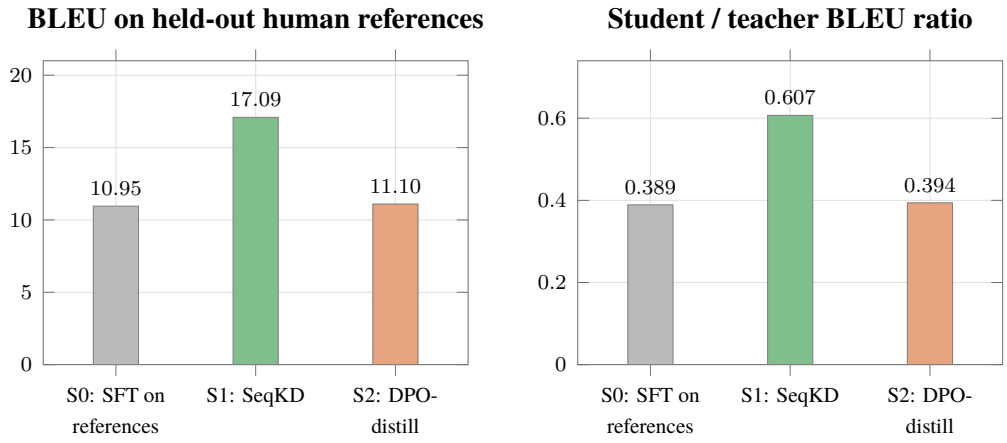


Figure 2.1: Head-to-head at 100,000 training sentences, Hindi→English, identical 52.6 M student, evaluated on held-out human references. SeqKD (S1) wins on both measures; DPO-distill (S2) is indistinguishable from the reference-trained baseline (S0)

H1 is refuted. Sequence-level distillation beats preference distillation by 6.0 BLEU, an improvement of about 54 %, and DPO applied on top of a reference-trained base adds 0.15 BLEU over that base, indistinguishable from noise. The replacement hypothesis, H1′, is supported by every subsequent result: *reference noise, not the training objective, is what caps a compact Indic student.* Teacher-distilled targets remove that noise, and the student then continues to improve with data: the ratio rises from 0.607 at 100,000 sentences to 0.764 at 250,000 and 0.806 at 500,000, whereas reference-trained students plateau near 0.66 regardless of data volume.

The DPO implementation is not defective, which is what makes the result informative. Its margin accuracy reaches 1.0, meaning it ranks preferred above dispreferred candidates perfectly; it consistently improves chrF by one to two points while costing about one BLEU point. The preference signal is being learned correctly; it simply is not the signal that a 52-million-parameter student most needs when its reference targets are noisy. SETU

3.2.2 Sequence Diagram

Figure 3.2 traces a single translation request. The significant branch is the pivot: when a directly trained model exists the engine performs one hop, and when it does not (every Indic-to-Indic pair) the engine performs two, passing the intermediate English string from the first student to the second. The response carries the summed latency of both hops and a `pivot` field so that the caller can tell the two cases apart.

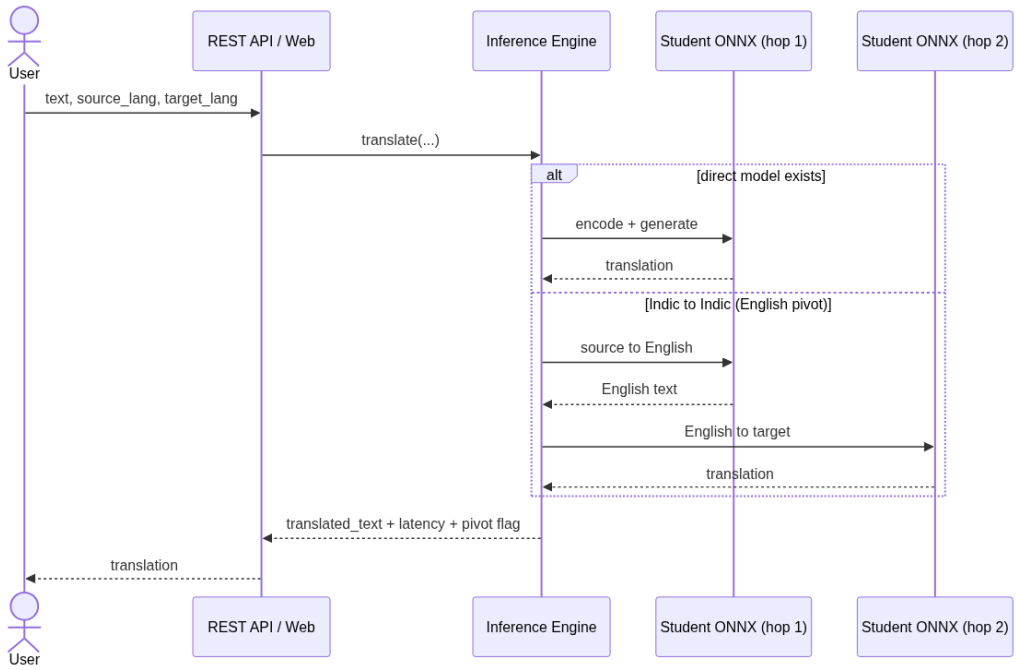


Figure 3.2: Sequence diagram: direct translation and the two-hop English pivot

3.2.3 Activity Diagram

Figure 3.3 shows the full training and deployment activity flow. Everything above the dividing line executes once per direction on a GPU; everything below executes on the user's device. The two decision points are the quality gate (does the student reach 80 % of teacher BLEU) and the routing decision at inference time.

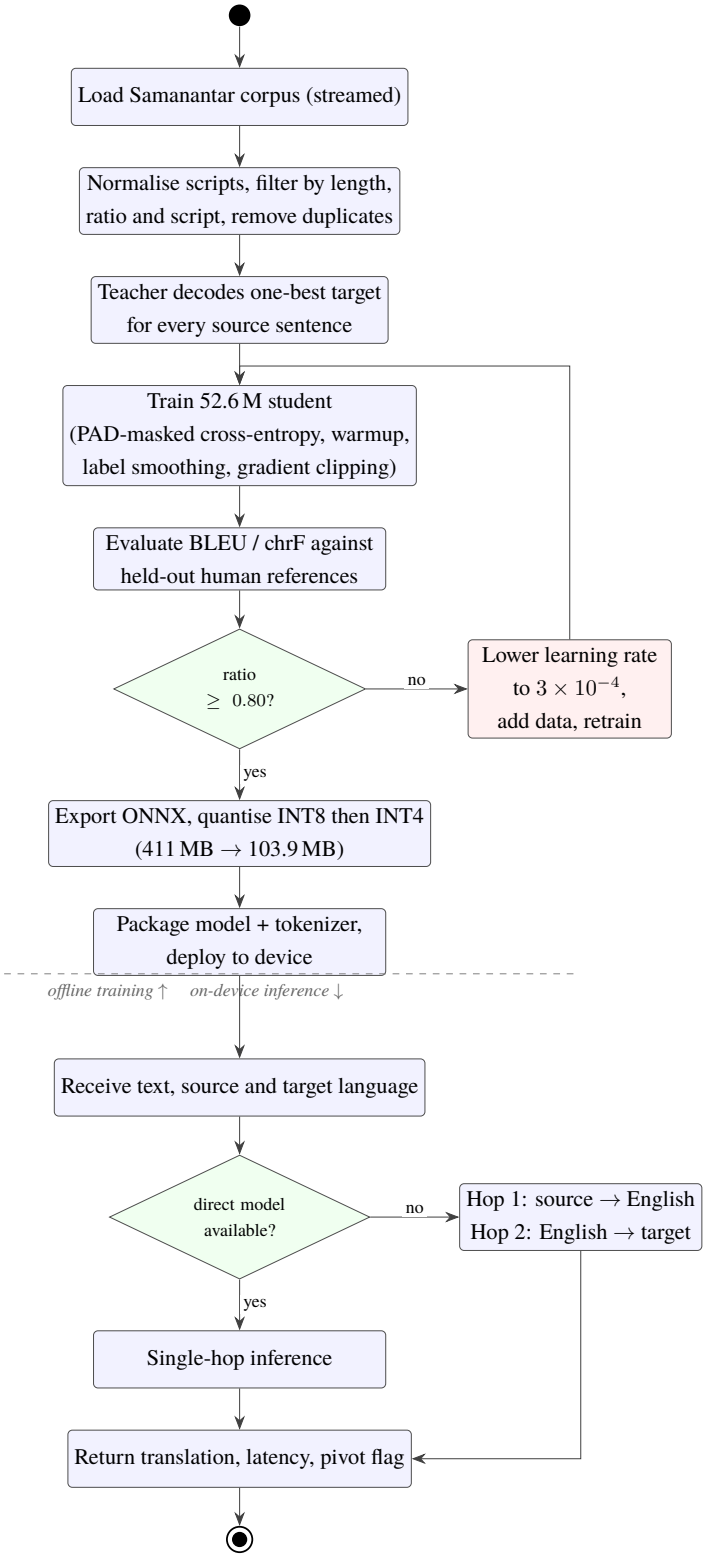


Figure 3.3: Activity diagram: offline training pipeline (above) and on-device inference (below)

3.2.4 State Chart Diagram

A model in SETU passes through a well-defined sequence of states. Figure 3.4 records them, including the two transitions that were exercised in practice: the collapse detected at evaluation and the recovery achieved by retraining at a lower learning rate on the same distilled corpus.

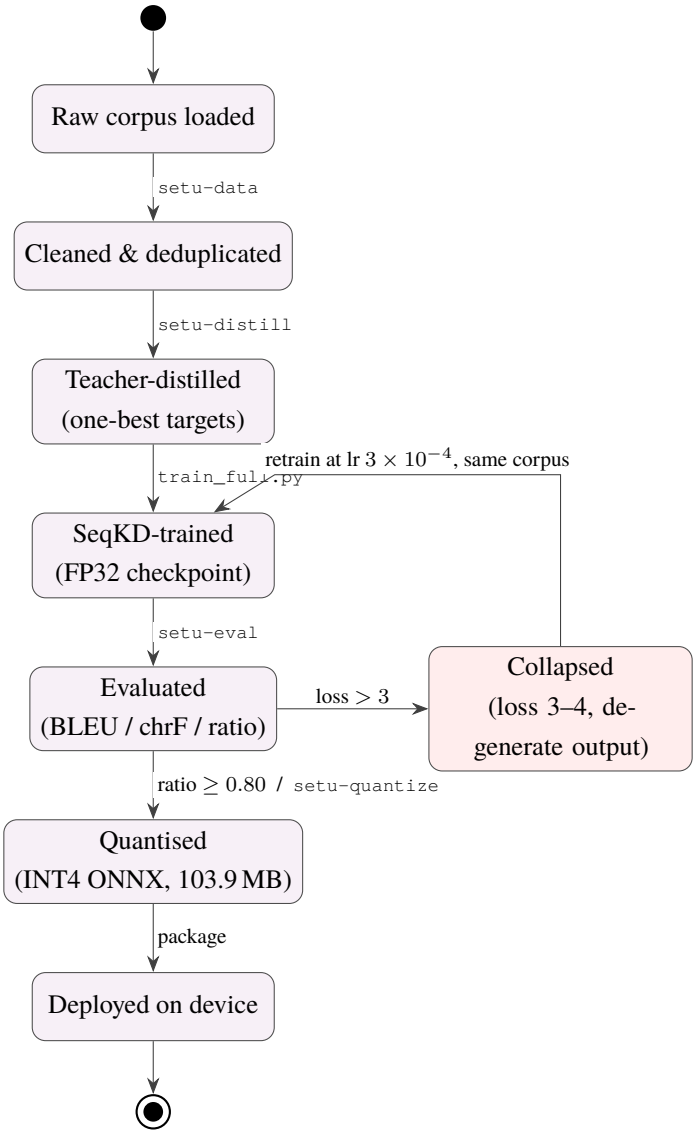


Figure 3.4: State chart diagram: model lifecycle from raw corpus to deployed artifact, including the observed collapse and recovery transitions

3.2.5 Data Flow Diagram

Figure 3.5 gives the level-1 data flow. The corpus is an external data source; the teacher is a transformation that produces the distilled data store; the trainer consumes that store and produces a student; the quantiser converts the student into a deployable artifact; the inference engine consumes the artifact and produces translations. The evaluator is a side branch that consumes both student and teacher outputs to compute the ratio that gates deployment.

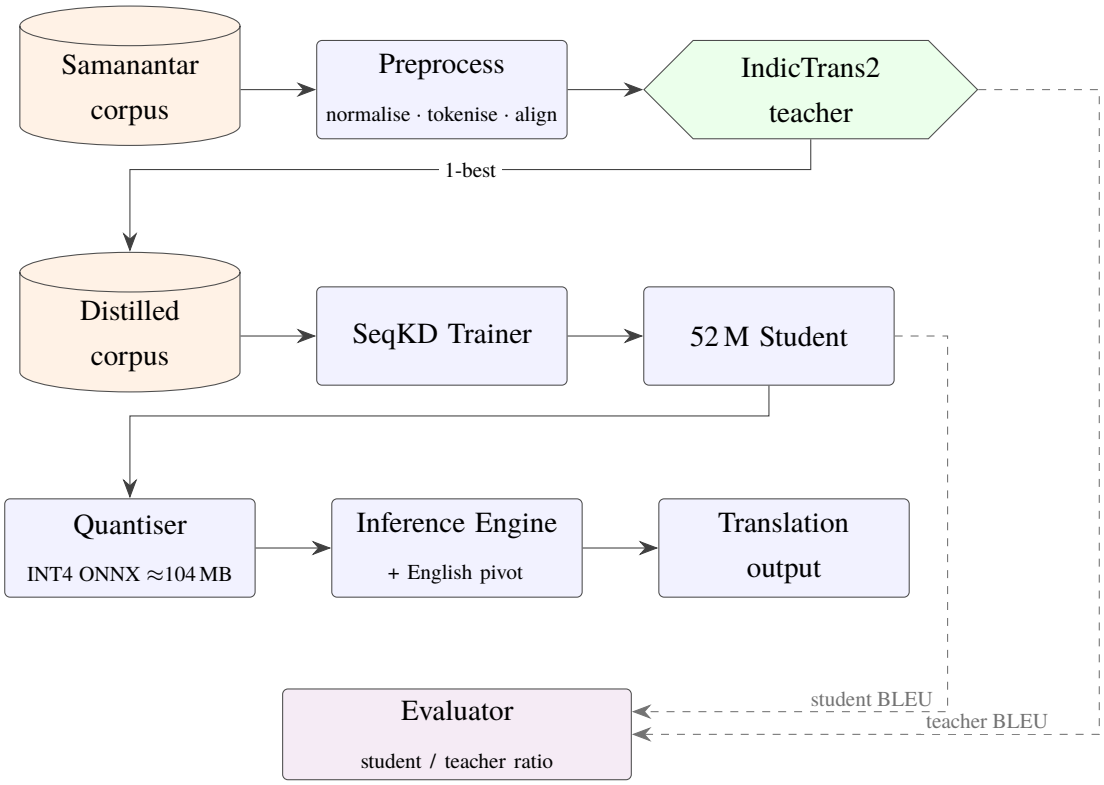


Figure 3.5: Level-1 data flow diagram: corpus to deployed translation

3.2.6 Entity-Relationship Model

SETU does not use a relational database (an offline edge deployment should not require one), but its persistent artifacts have a clear relational structure, recorded in Figure 3.6. Corpora are stored as JSONL, reports as JSON, models as ONNX with an accompanying SentencePiece tokenizer, and the language registry as YAML.

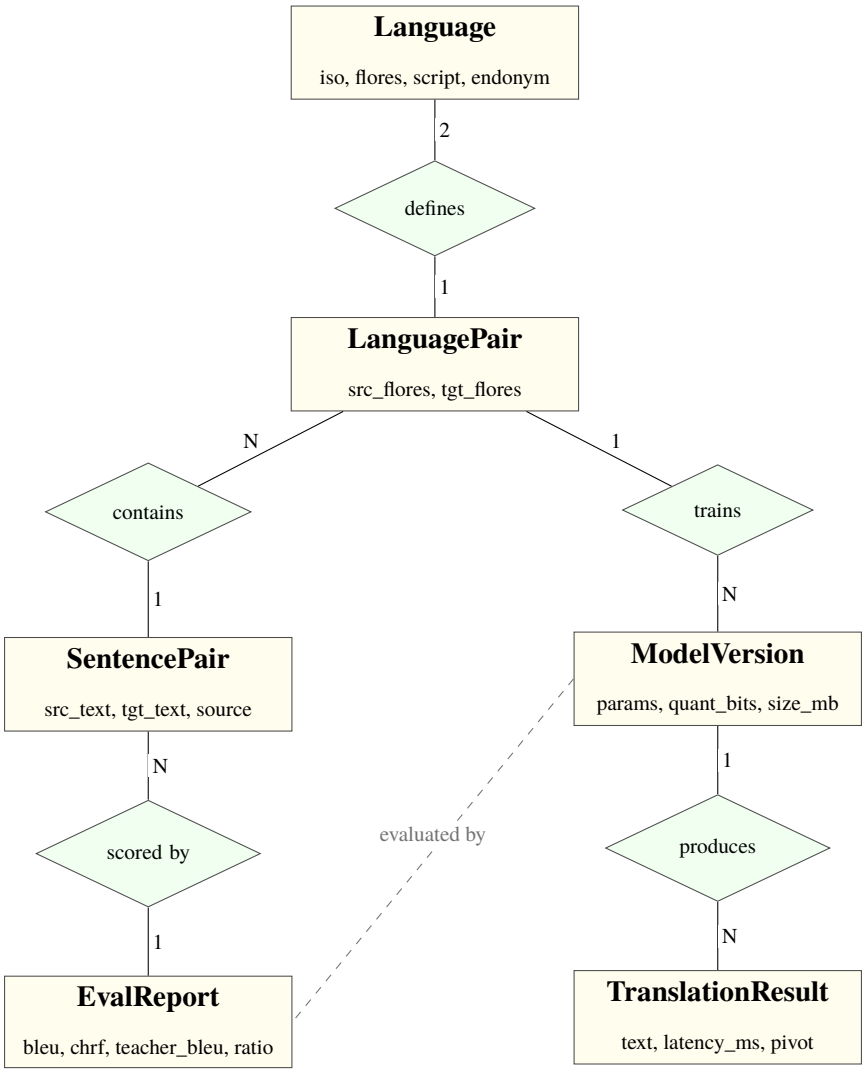


Figure 3.6: Entity-relationship model of SETU's persistent artifacts

3.2.7 Class Diagram

Figure 3.7 gives the principal classes. The design deliberately isolates every dependency on the teacher's implementation inside a single class, `IndicTrans2Teacher`. A unit test scans the source tree and fails the build if any other module imports `transformers`, `torch` or `IndicTransToolkit`. This is what allows the deployed artifact to contain no training dependencies at all.

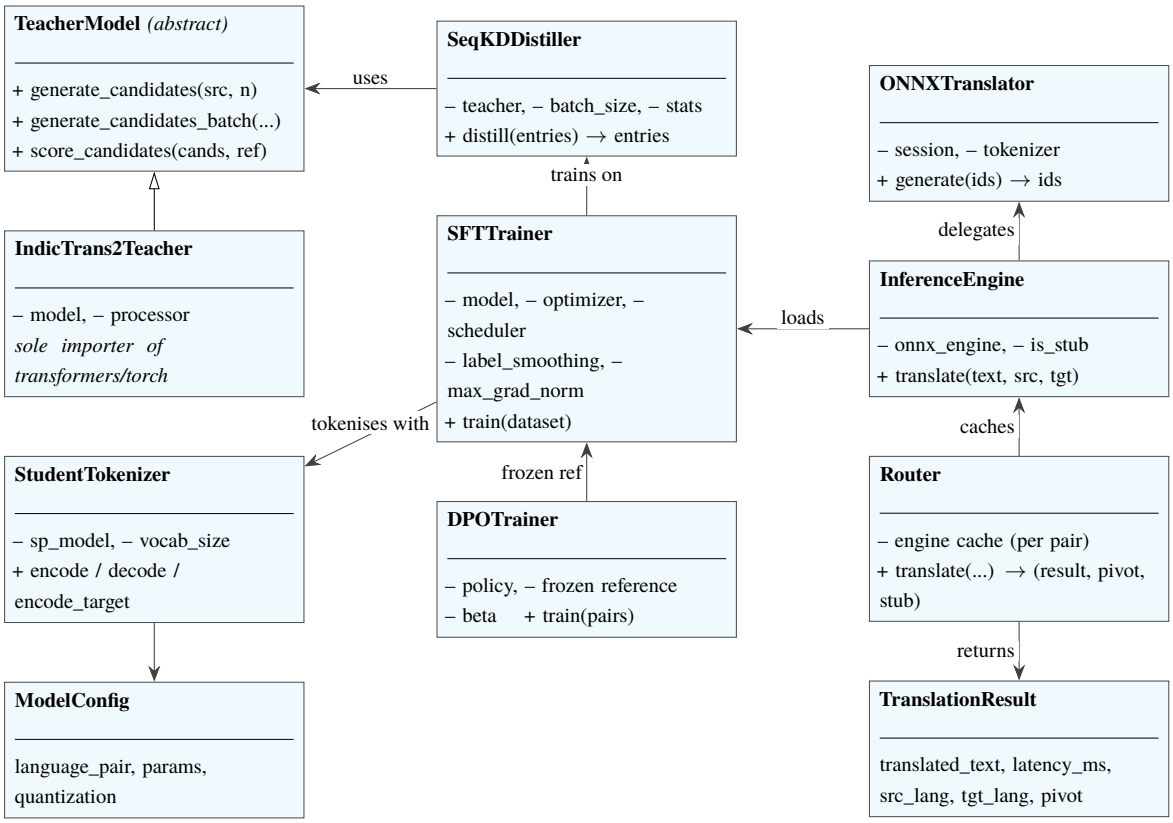


Figure 3.7: Class diagram: training classes (left) are used offline by the ML Engineer; inference classes (right) ship to the device

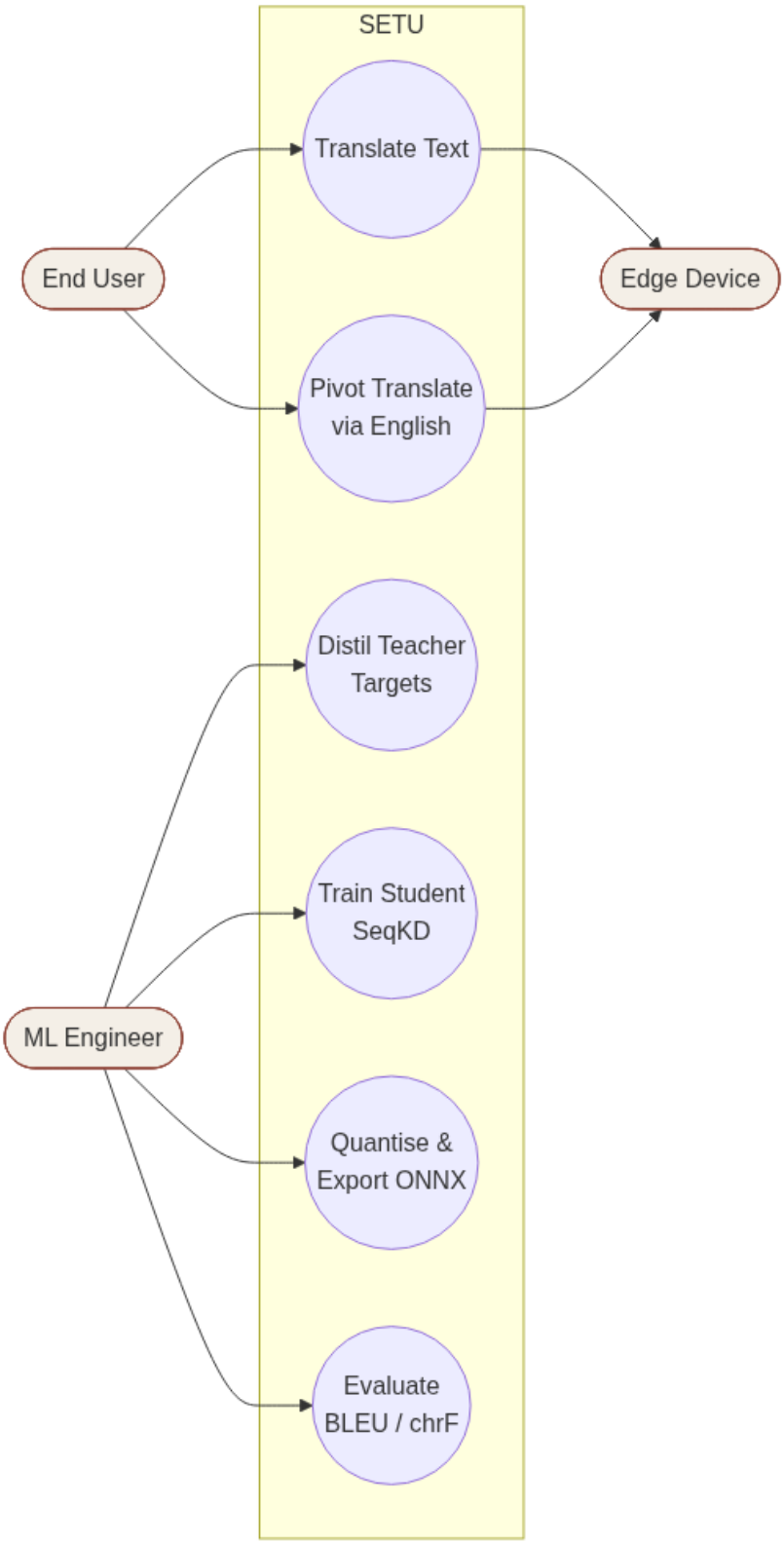


Figure 3.1: Use case diagram: End User, ML Engineer and Edge Device actors

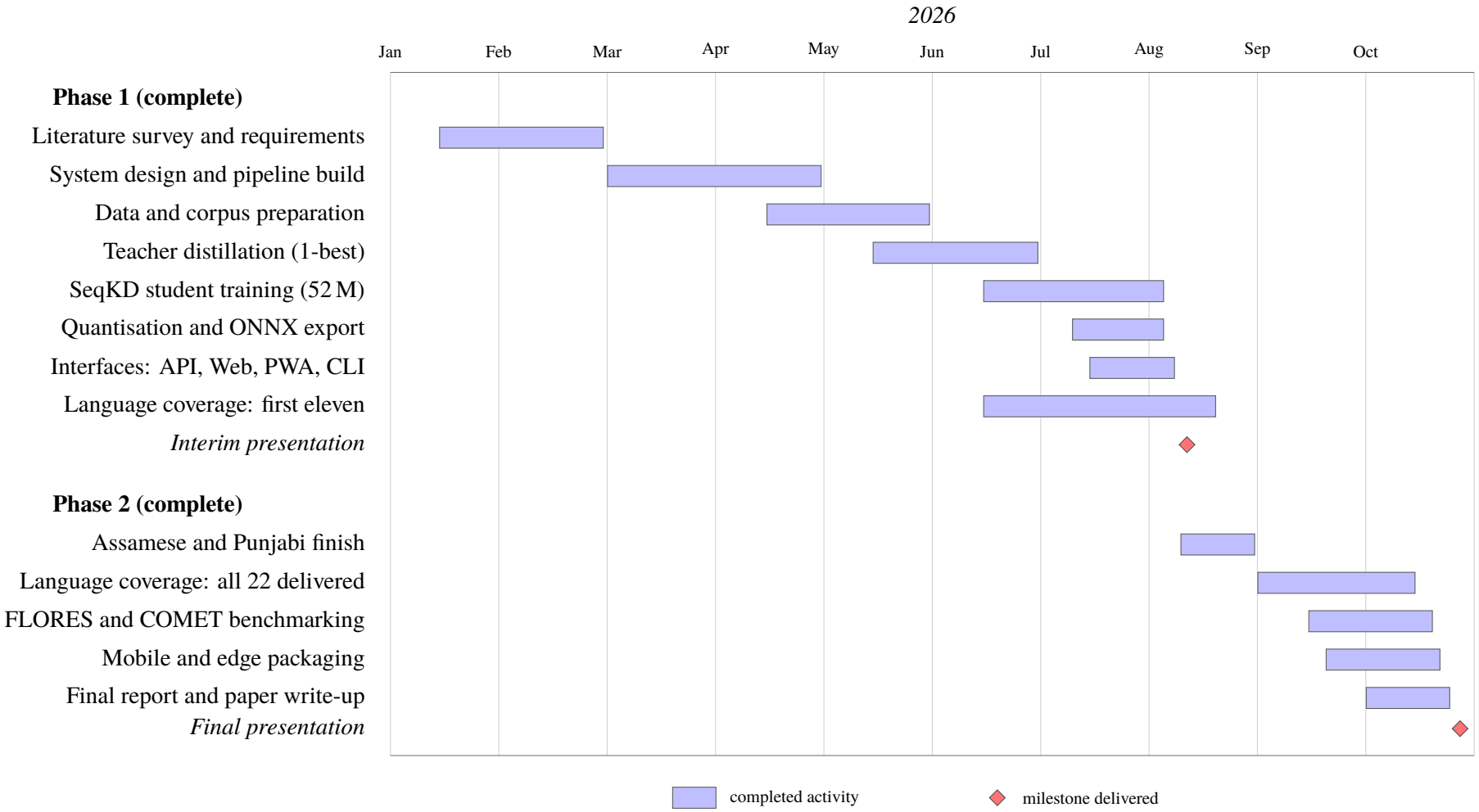


Figure 4.1: Gantt chart: both phases complete, all 22 scheduled languages delivered

the two collapses described in Section 8.4 consumed almost exactly the two weeks that activity G was allotted.

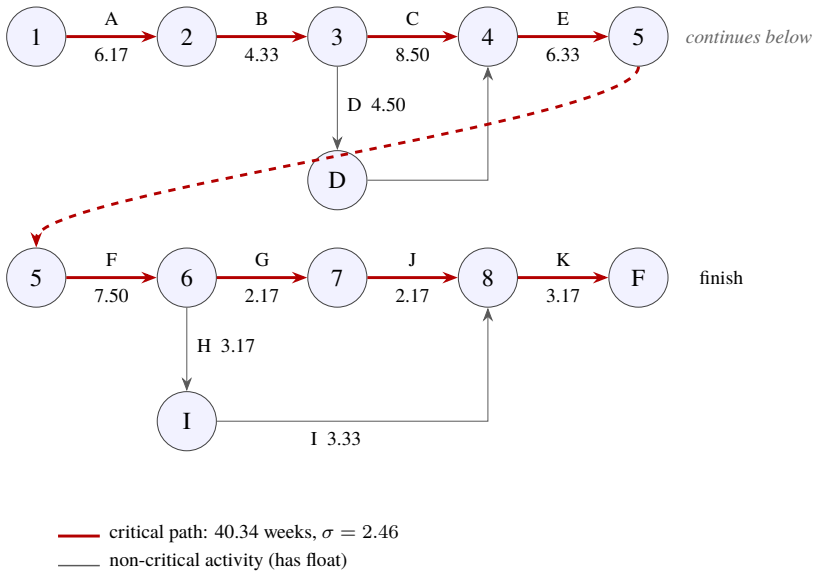


Figure 4.2: PERT network for Phase 1 with the critical path highlighted

4.1.4 Risk Management

Table 4.4 records the risks identified at planning time, together with the mitigation applied and (where the risk materialised) what actually happened. Five of the nine anticipated risks occurred.

Table 4.4: Risk register with outcomes

ID	Risk	Prob.	Impact	Mitigation and outcome
R1	Gated access to the teacher checkpoints and the BPCC corpus	High	High	Fall back to ungated rotary checkpoints and the Samanantar corpus. **Occurred.** Mitigation applied; the cost is a lower quality ceiling, reported explicitly
R2	No GPU available locally	High	High	Use a cloud GPU (Kaggle, Colab) and later a dedicated A40 VM. **Occurred.** All final training ran on the A40 VM

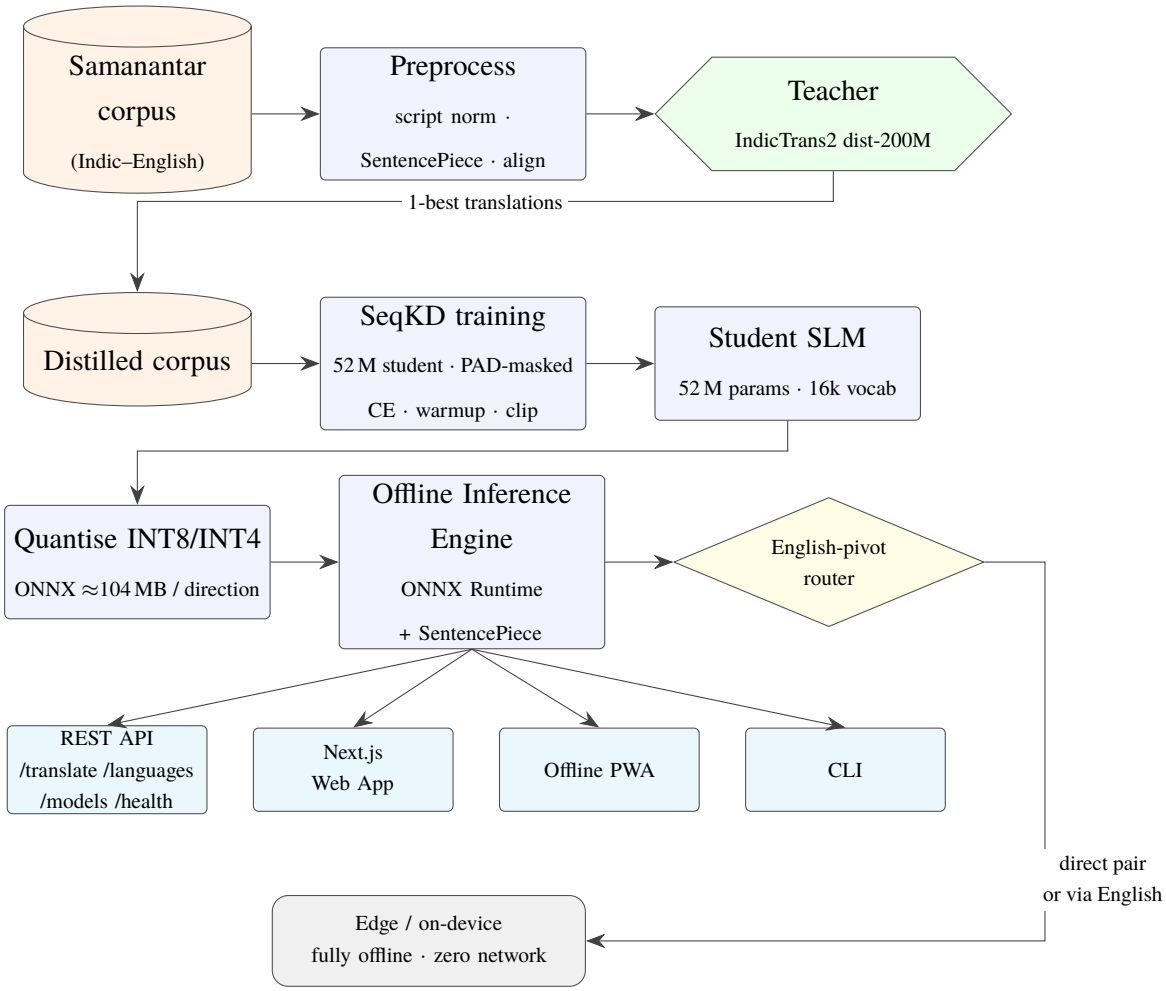


Figure 5.1: System architecture: SeqKD distillation and offline deployment pipeline

The architecture has four layers.

The data layer ingests parallel corpora, normalises and filters them, and stores two corpora per direction: the *processed* corpus, which retains human references and is used only for evaluation, and the *distilled* corpus, whose targets are the teacher's one-best translations and which is used for training. Keeping both is what allows evaluation to remain on real references regardless of what the student was trained on, the property that makes the comparison in Section 8.1 fair.

The model layer holds the teacher behind an abstract interface and the student behind a trainer. The teacher is used only offline, and only through three methods. The student is a Marian encoder–decoder trained from scratch.

The deployment layer converts a trained student into a device artifact: ONNX export, runtime validation, progressive INT8 and INT4 quantisation, and a per-stage benchmark. Its output is a self-contained directory.

The serving layer loads those directories on demand, caches one engine per direction, routes each request to a direct model or through the English pivot, and presents the

```python
class TeacherModel:
    """Abstract teacher. Only IndicTrans2Teacher may import
       transformers/torch."""

    def generate_candidates(self, src_text: str, src_lang: str,
                            tgt_lang: str, n: int = 4) -> list[str]:
        """Return n candidate translations, best first."""
        raise NotImplementedError

    def generate_candidates_batch(self, src_texts: list[str], src_lang: str,
                                  tgt_lang: str, n: int = 4) ->
                                      list[list[str]]:
        """Batched form -- the only method the SeqKD distiller needs."""
        raise NotImplementedError

    def score_candidates(self, candidates: list[str],
                         reference: str) -> list[float]:
        """Sentence-level chrF of each candidate against the reference."""
        raise NotImplementedError
```

Listing 6.2: The teacher interface (`src/setu/teacher/base.py`)

The wall is enforced mechanically. The test below walks the source tree and fails if any module other than the designated backend imports the teacher's dependencies. Without it the restriction would erode within weeks, and the deployed artifact would silently acquire a PyTorch dependency.

```python
FORBIDDEN = ("transformers", "torch", "IndicTransToolkit")
ALLOWED   = {"src/setu/teacher/indictrans2.py"}

def test_teacher_wall():
    """No module outside the backend may import the teacher's dependencies."""
    for path in Path("src/setu").rglob("*.py"):
        rel = path.as_posix()
        if rel in ALLOWED or "/training/" in rel or "/quantize/" in rel:
            continue                        # training and export legitimately
                use torch
        source = path.read_text()
        for name in FORBIDDEN:
            assert f"import {name}" not in source, f"{rel} imports {name}"
```

Listing 6.3: Enforcing the teacher wall (`tests/test_teacher.py`)

6.2.2 Corpus Cleaning

The cleaning filters encode domain knowledge about Indic text that a generic text pipeline would get wrong. Two details matter. Zero-width joiners and non-joiners must survive normalisation, because they distinguish valid conjuncts; a naive control-character strip corrupts correct text. And the script-fraction filter is what removes mined pairs in which one side is simply not in the claimed language.

```python
ZWJ, ZWNJ = "\u200d", "\u200c"   # zero-width joiner / non-joiner

def normalize(text: str) -> str:
    """NFC normalise and strip control characters, preserving ZWJ/ZWNJ."""
    text = unicodedata.normalize("NFC", text)
    return "".join(
        ch for ch in text
        if unicodedata.category(ch) != "Cc" or ch in (ZWJ, ZWNJ, "\n", "\t")
    )

def passes_length_ratio(src: str, tgt: str, max_ratio: float = 3.0) -> bool:
    """Reject misaligned pairs: mined corpora pair unrelated sentences."""
    if not src or not tgt:
        return False
    ratio = max(len(src), len(tgt)) / min(len(src), len(tgt))
    return ratio <= max_ratio

def passes_script_fraction(text: str, script: str, minimum: float = 0.5) ->
    bool:
    """At least `minimum` of the letters must belong to the expected
        script."""
    letters = [ch for ch in text if ch.isalpha()]
    if not letters:
        return False
    in_script = sum(1 for ch in letters if script_of(ch) == script)
    return in_script / len(letters) >= minimum
```

Listing 6.4: Corpus filters (`src/setu/corpus/filters.py`)

6.2.3 The SeqKD Distiller

The distiller is short, which is appropriate: sequence-level distillation is a simple idea, and the project's contribution is the evidence that it is the right one, not the code that implements it.

```python
class SeqKDDistiller:
    """Replace each entry's human target with the teacher's 1-best
```

```
        translation."""

    def __init__(self, teacher: TeacherModel, batch_size: int = 32):
        self.teacher = teacher
        self.batch_size = batch_size
        self.stats = {"entries": 0, "empty": 0}

    def distill(self, entries: list[CorpusEntry]) -> list[CorpusEntry]:
        out: list[CorpusEntry] = []
        for start in range(0, len(entries), self.batch_size):
            chunk = entries[start:start + self.batch_size]
            cands = self.teacher.generate_candidates_batch(
                [e.src_text for e in chunk],
                chunk[0].src_lang, chunk[0].tgt_lang, n=1,
            )
            for e, c in zip(chunk, cands):
                if c and c[0].strip():
                    tgt = c[0]                  # the teacher's target
                        replaces the human one
                else:
                    tgt = e.tgt_text            # safe fallback, never a
                        blank target
                    self.stats["empty"] += 1
                out.append(CorpusEntry(e.src_lang, e.tgt_lang,
                                       e.src_text, tgt, "seqkd"))
            self.stats["entries"] += len(chunk)
        return out
```

Listing 6.5: Sequence-level distillation (`src/setu/distill/distiller.py`)

The `source` field is set to `"seqkd"` so that a distilled corpus can never be mistaken for a reference corpus downstream. The evaluation code asserts that it is scoring against entries whose source is *not* `seqkd`, which is the mechanical guarantee behind the fairness of Section 8.1.

6.2.4 The Training Loop

This is the most-debugged code in the project. Every line commented below corresponds to a failure that was observed and diagnosed; the run history is in Section 8.3.

```
def train(self, dataset) -> dict:
    total_steps = len(dataloader) * self.epochs
    # Warmup was previously read from config and silently ignored; a constant
        LR
    # underfits a from-scratch transformer. Cap warmup at 10% of the run.
```

```
    warmup = min(self.config.get("warmup_steps", 0), max(1, total_steps //
        10))
    scheduler = get_linear_schedule_with_warmup(self.optimizer, warmup,
        total_steps)

    for epoch in range(self.epochs):
        for batch in dataloader:
            outputs = self.model(input_ids=batch["input_ids"],
                                 attention_mask=batch["attention_mask"],
                                 labels=batch["labels"])   # labels use -100
                                     at PAD
            if self.label_smoothing > 0:
                loss = F.cross_entropy(
                    outputs.logits.view(-1, outputs.logits.size(-1)),
                    batch["labels"].view(-1),
                    ignore_index=-100,                  # never score padding
                    label_smoothing=self.label_smoothing,
                )
            else:
                loss = outputs.loss
            loss.backward()
            # The single most important stabiliser: without clipping, a
                200k-sentence
            # run collapsed to constant output while 120k on the same code
                succeeded.
            torch.nn.utils.clip_grad_norm_(self.model.parameters(),
                self.max_grad_norm)
            self.optimizer.step()
            scheduler.step()
            self.optimizer.zero_grad()
```

Listing 6.6: Supervised training step with all four stabilisers (`src/setu/training/sft.py`)

The dataset builds the labels, and the single most consequential line in the entire training path is the padding mask:

```
def encode_target(self, text: str) -> list[int]:
    """Labels are [tokens, EOS] with NO leading BOS: HuggingFace already
        prepends
    the decoder start token, so adding one wastes a decode step and creates a
    train/generate mismatch that produced empty output in an early run."""
    ids = self.sp.encode(text)[: self.max_seq_len - 1]
    return ids + [self.eos_id]

def collate(self, batch: list[dict]) -> dict:
```

```
    labels = pad_sequence([b["labels"] for b in batch], batch_first=True,
                          padding_value=PAD_ID)
    # -100, not PAD: HuggingFace's loss uses ignore_index=-100. Padding with
        PAD
    # gave the model free credit for predicting PAD, producing a fake-low
        loss of
    # 0.274 and a decoder that ignored the source entirely.
    labels[labels == PAD_ID] = -100
    return {"input_ids": ..., "attention_mask": ..., "labels": labels}
```

Listing 6.7: Label construction (`src/setu/training/dataset.py`)

6.2.5 The Inference Router

The router is the whole of Setu's multilingual coverage story. It is also the one place where routing logic exists, which is what prevents the interfaces from diverging.

```
@lru_cache(maxsize=None)
def engine_for_pair(pair: str) -> InferenceEngine:
    """One cached engine per pair; a pair with no model on disk stays a
        stub."""
    return InferenceEngine(ModelConfig(language_pair=pair))

def translate(text, src_lang, tgt_lang) -> tuple[TranslationResult, str |
    None, bool]:
    """Returns (result, pivot, stub). Raises ValueError on unknown or equal
        codes."""
    src, tgt = resolve_language(src_lang), resolve_language(tgt_lang)
    if src["iso"] == tgt["iso"]:
        raise ValueError(f"Source and target language are both
            {src['iso']!r}")

    direct = f"{src['flores']}-{tgt['flores']}"
    if _available(direct):
        return engine_for_pair(direct).translate(text, src["iso"],
            tgt["iso"]), None, False

    # No direct model: chain source -> English -> target if both halves exist.
    eng = load_languages()["en"]["flores"]            # eng_Latn
    if src["iso"] != "en" and tgt["iso"] != "en":
        hop1, hop2 = f"{src['flores']}-{eng}", f"{eng}-{tgt['flores']}"
        if _available(hop1) and _available(hop2):
            r1 = engine_for_pair(hop1).translate(text, src["iso"], "en")
            r2 = engine_for_pair(hop2).translate(r1.translated_text, "en",
                tgt["iso"])
```

```
            return (TranslationResult(
                        translated_text=r2.translated_text,
                        src_lang=src["iso"], tgt_lang=tgt["iso"],
                        latency_ms=(r1.latency_ms or 0.0) + (r2.latency_ms or
                            0.0)),
                    "en", False)
    # Nothing can serve this pair: pass through, but say so.
    return engine_for_pair(direct).translate(text, src["iso"], tgt["iso"]),
        None, True
```

Listing 6.8: English-pivot routing (`src/setu/inference/router.py`)

6.2.6 Offline Verification

The offline guarantee is a claim about the absence of behaviour, which cannot be established by inspection alone. It is therefore tested by removing the capability entirely: all socket construction is patched to raise, and translation is then required to succeed.

```
def assert_offline():
    """Disable every socket so any network call raises instead of
        succeeding."""
    def _blocked(*args, **kwargs):
        raise RuntimeError("network access attempted during offline
            inference")
    socket.socket = _blocked
    socket.create_connection = _blocked

def test_onnx_engine_translates_offline(tmp_models):
    assert_offline()                          # no sockets from here on
    engine = InferenceEngine(ModelConfig(language_pair="hin_Deva-eng_Latn"))
    result = engine.translate("bhaarat ek vishaal desh hai.", "hi", "en")
    assert result.translated_text                       # non-empty
    assert not engine.is_stub                           # a real model
        answered
```

Listing 6.9: Proving offline operation (`tests/test_quantize.py`)

6.2.7 The REST Service

```
@app.post("/translate", response_model=TranslateResponse)
def translate(req: TranslateRequest) -> TranslateResponse:
    try:
        result, pivot, stub = router.translate(req.text, req.source_lang,
            req.target_lang)
```

```
    except ValueError as exc:                # unknown code, or src == tgt
        raise HTTPException(status_code=400, detail=str(exc))
    return TranslateResponse(
        translated_text=result.translated_text,
        source_lang=req.source_lang, target_lang=req.target_lang,
        latency_ms=result.latency_ms,
        pivot=pivot,      # "en" when routed through English, else null
        stub=stub,        # true means passthrough: never mistake it for a
            translation
    )

@app.get("/models")
def models() -> dict:
    """Discover every model directory on disk with its variant and size."""
    return {"count": len(_scan_models()), "models": _scan_models()}
```

Listing 6.10: Translation endpoint (`interfaces/rest/app.py`)

6.2.8 Code Efficiency and Optimisation

Efficiency in SETU means fitting three budgets: training memory, artifact size and inference latency. Table 6.1 records the optimisations applied and their measured effect.

Table 6.1: Optimisations applied and their measured effect

Optimisation	Problem addressed	Measured effect
Progressive INT8 then INT4 quantisation	410.9 MB FP32 artifact is over twice the 200 MB budget	410.9 → 104.6 → 103.9 MB, a 4.0× compression, with negligible quality loss
Greedy decoding at deployment	Beam search costs roughly 4× the latency	p90 latency falls to 141–350 ms for most directions; beam search retained for offline evaluation only
Corpus streaming	The full Samanantar corpus is tens of gigabytes	Only the requested slice is ever materialised
Greedy teacher decoding for distillation	Beam-5 distillation of 500,000 sentences does not fit a session	4–5× faster; target quality unaffected
Per-pair engine cache (`lru_cache`)	Reloading a 104 MB ONNX session per request	Each direction loads from disk at most once

Optimisation	Problem addressed	Measured effect
Free the teacher before training	Teacher plus student exceeded 15 GB of GPU memory	Eliminated the CUDA out-of-memory failure in the first GPU run
Batch size 32 for SFT, 16 for DPO	DPO holds policy and reference and runs four forward passes per step	Both fit a 15–16 GB accelerator
16,000-token vocabulary	A 32,000-token vocabulary left most tokens undertrained	Eliminated degenerate decoding; smaller embedding matrix
Sequence truncation at `max_seq_len`	Sentences beyond the positional-embedding limit crashed training and evaluation	Fixed the crash; a regression test prevents recurrence
Resumable stage markers	A dropped SSH session discarded hours of work	Interrupted runs resume at the incomplete stage
`expandable_segments` disabled	The CUDA virtual-memory allocator is unsupported on a vGPU slice	Removed the immediate crash on first allocation

6.2.8.1 The Quantisation Trade-off, Measured

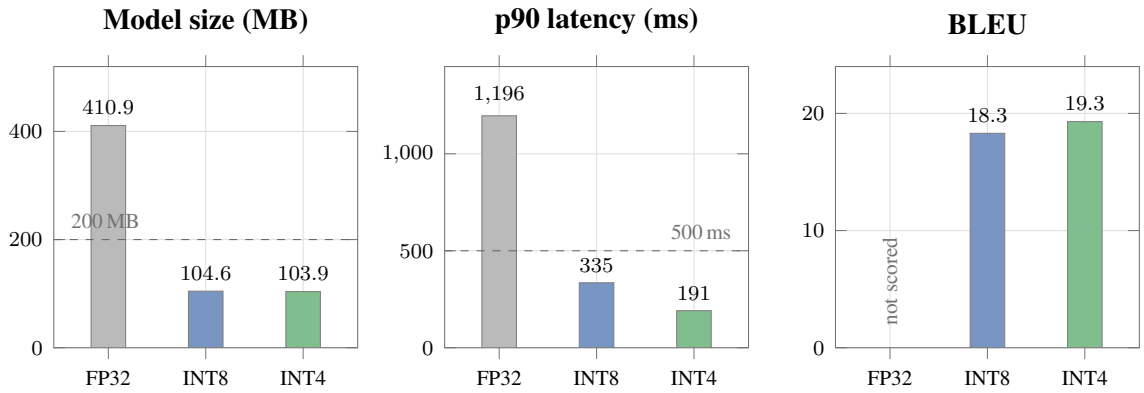


Figure 6.1: Effect of quantisation on size, latency and quality (Hindi→English student). INT4 is the shipped variant: it is the only precision that clears both the 200 MB and the 500 ms budget, at no measurable cost in BLEU

The arrows record an important intermediate finding. The first quantisation run produced latencies above the 500 ms budget not because quantisation was ineffective but because the

7.1.2 Unit Testing

Unit tests exercise one module in isolation. The tables below give the cases for the components whose failure modes matter most, each with at least one valid and two invalid inputs.

7.1.2.1 Translation Engine

Table 7.2: Unit test UT-1 to UT-5: translation engine, Hindi→English INT4 ONNX student

#	Test data (input)	Expected result	Actual result	Verdict
UT-1	`src=hi, tgt=en`, भारत एक विशाल देश है।	Fluent English translation	*"India is a vast country."*	**Pass**
UT-2	`src=hi, tgt=hi` (identical languages)	Rejected with HTTP 400 / `ValueError`	HTTP 400: *source and target are both 'hi'*	**Pass**
UT-3	`src=xx, tgt=en` (unknown language code)	Rejected: unknown language code	`ValueError` raised; request refused	**Pass**
UT-4	Empty string input	Empty output, no exception	`""` returned; no exception	**Pass**
UT-5	Sentence exceeding `max_seq_len`	Truncated and translated, not crashed	Truncated to the limit; translation returned	**Pass**

7.1.2.2 Corpus Pipeline

Table 7.3: Unit tests UT-6 to UT-10: corpus cleaning filters

#	Test data (input)	Expected result	Actual result	Verdict
UT-6	Well-formed Hindi–English pair	Retained	Retained	**Pass**
UT-7	Pair with length ratio 5:1	Rejected (ratio > 3.0)	Rejected, counted as `length_ratio`	**Pass**
UT-8	Devanagari source paired with a Tamil "English" side	Rejected (script fraction below 0.5)	Rejected, counted as `wrong_script`	**Pass**
UT-9	Text containing ZWJ and ZWNJ	Both preserved through normalisation	Both preserved; NFC applied	**Pass**
UT-10	Duplicate of an earlier pair	Removed	Removed, counted as `duplicate`	**Pass**

7.1.2.3 Training Mechanics

These are the property tests. Each asserts something that cannot be verified by reading the output of a training run.

Table 7.4: Unit tests UT-11 to UT-15: training mechanics

#	Property under test	Expected result	Actual result	Verdict
UT-11	DPO reference model is frozen	No parameter of the reference receives a gradient	All reference gradients `None`	**Pass**
UT-12	Training loss decreases over a short run	Final loss below initial	Loss decreased monotonically	**Pass**
UT-13	Padding is excluded from the loss	Label tensor contains -100 at every pad position	Verified for every batch	**Pass**
UT-14	EWC penalty is zero at the snapshot	Penalty $= 0$ at θ^*, > 0 away from it	0.0 at snapshot; grows with distance	**Pass**
UT-15	AIO-KD samples every direction	Coverage report lists all configured directions	All directions sampled	**Pass**

7.1.3 Integration Testing

Integration tests exercise two or more modules together. The most important is the English pivot, because it is the only path in the system in which the output of one model becomes the input of another.

7.1.3.1 Inference Engine and Pivot Router

Table 7.5: Integration tests IT-1 to IT-4: English-pivot chaining for Indic-to-Indic pairs

#	Test data (input)	Expected result	Actual result	Verdict
IT-1	`src=hi, tgt=bn`, भारत एक विशाल देश है। (no direct model)	Route via English; return Bengali	ভারত একটি বিশাল দেশ। (`pivot="en"`, ≈57 ms)	**Pass**
IT-2	`src=ta, tgt=te`, இந்தியா ஒரு பெரிய நாடு.	Route via English; return Telugu	భారతదేశం ఒక పెద్ద దేశం. (`pivot="en"`)	**Pass**
IT-3	`src=as, tgt=bn`, ভাৰত এখন ডাঙৰ দেশ।	Route via English; return Bengali	ভারত একটি বড় দেশ। (`pivot="en"`)	**Pass**
IT-4	`src=mr, tgt=kn`	Route via English; return Kannada	Correct Kannada returned, ≈50–65 ms	**Pass**

7.1.3.2 REST API and Inference Engine

Table 7.6: Integration tests IT-5 to IT-9: REST service against the routing layer

#	Request	Expected result	Actual result	Verdict
IT-5	`GET /models`	All installed pairs with variant and size	`count: 22`, each `int4`, 103.9 MB	**Pass**
IT-6	`GET /languages`	22 scheduled languages plus English	23 entries with ISO, FLORES, script, endonym	**Pass**
IT-7	`POST /translate` with `ta→en`	Real translation, `stub:false`	Correct English, ≈40–60 ms, `stub:false`	**Pass**
IT-8	`POST /translate` with an unknown code	HTTP 400 with a descriptive message	HTTP 400 returned	**Pass**
IT-9	CORS preflight from the browser client	Preflight accepted	`OPTIONS` returns the expected headers	**Pass**

7.1.3.3 Full Pipeline

Table 7.7: Integration tests IT-10 to IT-13: end-to-end pipeline

#	Scenario	Expected result	Actual result	Verdict
IT-10	Corpus → distil → train → evaluate on a small slice	Runs end to end; a training report is written	Report written with BLEU, chrF and ratio	**Pass**
IT-11	Train → export → quantise → translate offline	Quantised model translates with sockets disabled	Translation returned; no network call	**Pass**
IT-12	Same pipeline on a non-default language pair	Pair-agnostic: succeeds without code change	Tamil→English and reverse both succeeded	**Pass**
IT-13	CLI and REST for the same request	Identical output from both interfaces	Identical, since both call the same router	**Pass**

IT-13 deserves a note, because it was added in response to a genuine defect. The CLI originally constructed an `InferenceEngine` with the language pair from configuration rather than from the request, so `--src as` loaded the Hindi model and translated with it. The output looked like a translation and was wrong. The fix, routing both interfaces through `setu.inference.router`, is exactly what this test now enforces.

7.1.4 System and Non-Functional Testing

Table 7.8: System tests ST-1 to ST-6: non-functional targets

#	Attribute	Target	Measured	Verdict
ST-1	Artifact size	≤ 200 MB per direction	103.9 MB for all 22 directions	**Pass**
ST-2	Inference latency	< 500 ms per sentence	Mean 105–202 ms; p90 141–541 ms	**Pass**(one p90 outlier)
ST-3	Offline operation	No network call at inference	All sockets disabled; translation succeeded	**Pass**
ST-4	Quality ratio	≥ 0.80 of teacher BLEU	14 of 22 directions; mean 0.838	Partial
ST-5	Model health across all directions	Every model produces non-degenerate output	22 of 22 healthy; no empty, passthrough or collapsed output	**Pass**
ST-6	Coverage	All pairs among supported languages reachable	22 direct plus English-pivoted routes	**Pass**

ST-5 was implemented as a dedicated script, `scripts/translation_suite.py`, which loads each model in turn, translates a real source sentence in that model's own direction, and flags three specific pathologies: empty output, output identical to the input (a passthrough masquerading as a translation), and collapsed output (the same string returned for different inputs). It is the test that confirmed the two recovered models were genuinely repaired rather than merely scoring better.

7.1.5 Defects Found by Testing

Table 7.9 lists the defects the suite caught, with the test that caught each. They are reported because the class of defect is instructive: most were silent, producing plausible output rather than an error.

Table 8.1: The four systems compared

System	SFT targets	DPO?	Rationale
S0	Human references	No	Naive baseline: train on the data as given
S1	Teacher one-best	No	The standard strong baseline [21]
S2	Human references	Yes	The project's original proposal
S3	Teacher one-best	Yes	Does preference learning help on top of strong distillation?

8.1.2 Result

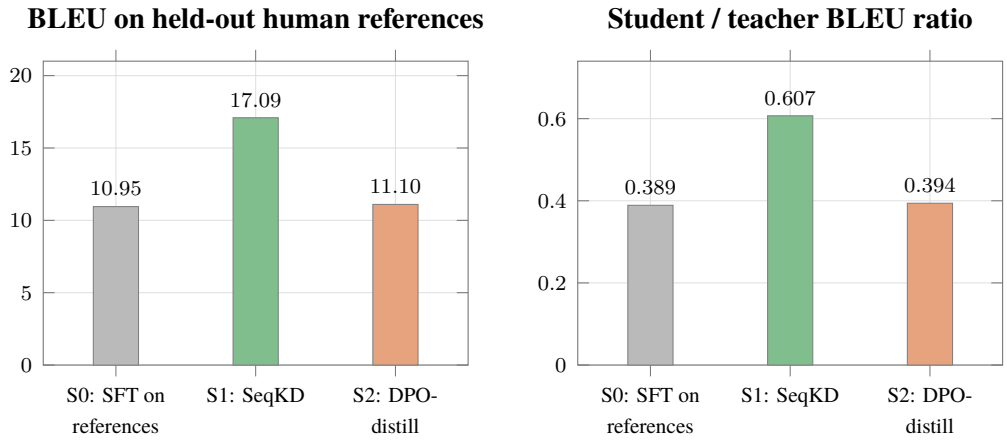


Figure 8.1: Head-to-head at 100,000 sentences, Hindi→English, matched student size, evaluated on human references (teacher BLEU 28.16). SeqKD (S1) gains $+6.14$ BLEU over the reference-trained baseline S0; DPO-distill (S2) gains $+0.15$

The hypothesis is refuted, and not marginally. Sequence-level distillation beats preference distillation by **5.99 BLEU**, an improvement of roughly **54 %**, at identical model size on identical data. Applying DPO on top of a reference-trained base adds 0.15 BLEU over that base, within the noise of a 500-sentence evaluation set.

8.1.3 Why the Hypothesis Failed

The most important thing about this result is that the DPO implementation is not defective. Three pieces of evidence establish that.

8.2 Data Scaling of Sequence-Level Distillation

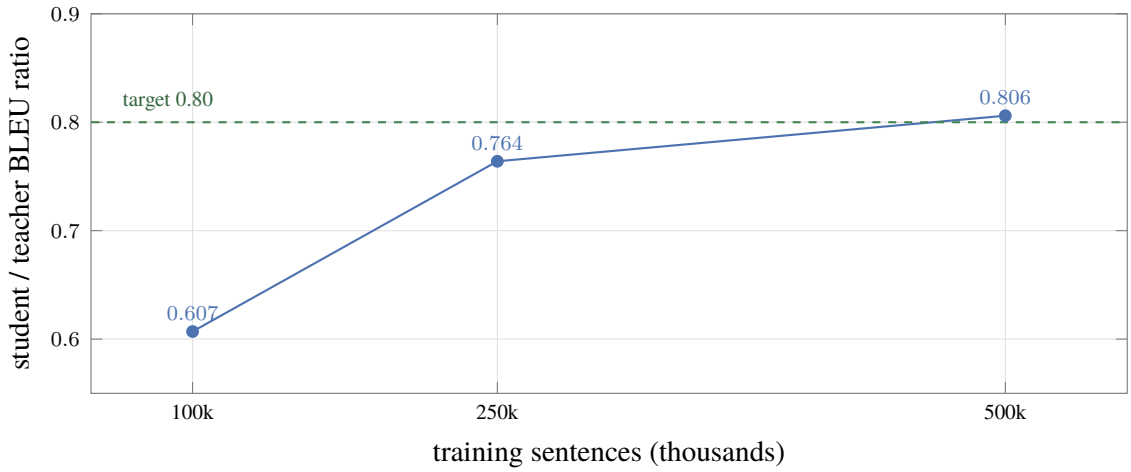


Figure 8.2: SeqKD data-scaling curve, Hindi→English. The ratio rises steeply to 250,000 sentences and then flattens as the student approaches its teacher's ceiling, crossing the 0.80 target at 500,000

The curve rises steeply from 100,000 to 250,000 sentences and then flattens, which is the expected shape as the student approaches its teacher's ceiling. The ungated distilled-200M teacher itself scores only 27–29 BLEU on these slices, so a ratio of 0.806 means the student is producing about four fifths of the quality of a teacher that is itself substantially weaker than the gated flagship. The practical consequence, discussed in Chapter 9, is that the most effective remaining lever is not more data but a stronger teacher.

The absolute BLEU values at 250,000 and 500,000 sentences (22.10 and 21.56) appear to move in the wrong direction. They do not: the teacher's own score differs between the two evaluation slices (28.91 against 27.08), which is why the ratio, not the raw BLEU, is the metric the project gates on. This slice-dependence is precisely why FLORES-200 rescoring is listed as outstanding work.

8.3 Training Run History and Failure Analysis

Three distinct failure modes were encountered before the first successful training run. Each is reported because each was diagnosed to a root cause and fixed, and because all three produce output that looks superficially reasonable.

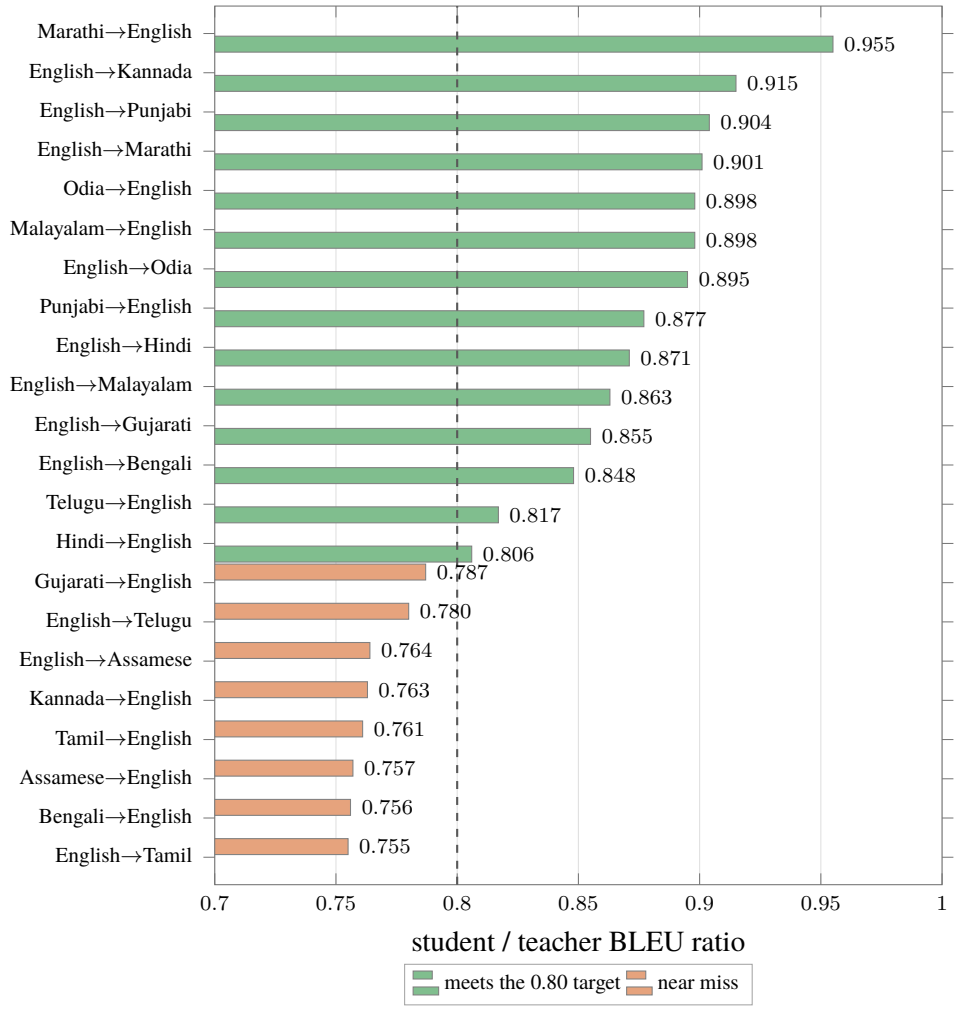


Figure 8.3: Student/teacher BLEU ratio for all 22 evaluated directions on the 500-sentence Samanantar development slice. Fourteen clear the 0.80 target; the dashed line marks the threshold

8.5.2 Per-Language Rollup and Quality Tiers

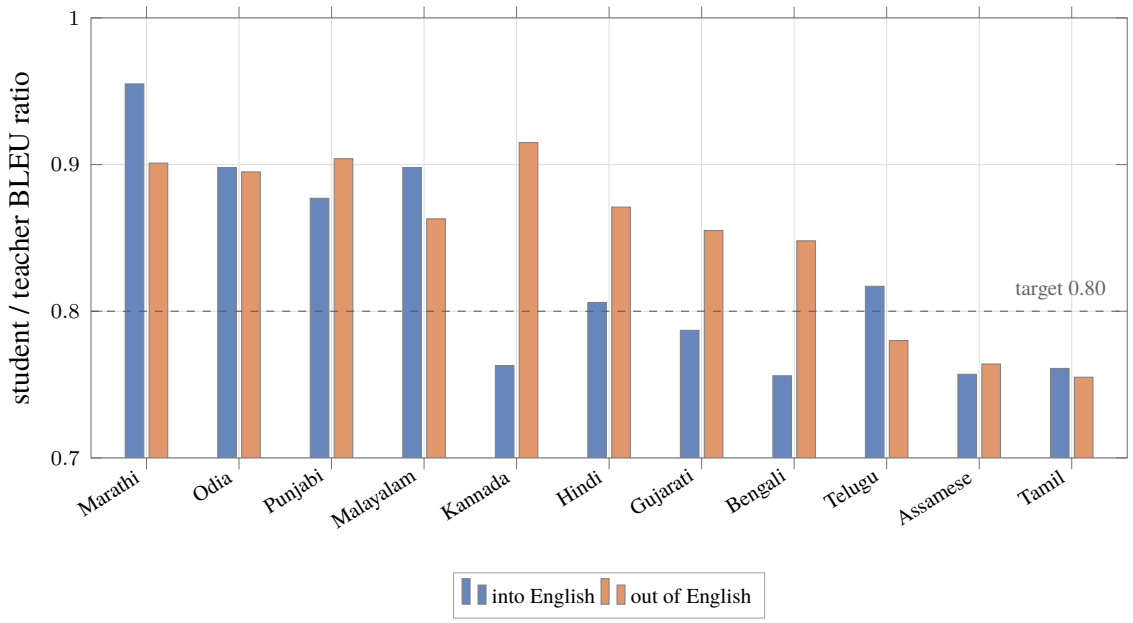


Figure 8.4: Per-language ratio in both directions for all eleven languages, ordered by their two-direction mean. The two directions track closely, so the method is not biased towards translating into English

Three observations follow.

Direction symmetry holds. Across all eleven languages, English→Indic averages 0.850 and Indic→English averages 0.825. The two directions are close, which suggests the method is not biased towards translating into English, a common asymmetry in multilingual systems.

The near-misses are under-converged, not unstable. All eight near-miss directions have supervised losses of 2.00 to 2.32, at the upper end of the healthy band of 1.5 to 2.3, whereas the passing directions cluster between 1.46 and 1.95. The pattern points to insufficient training rather than instability, which means the cheap remedies (beam search at evaluation, more epochs, more distilled data) are the appropriate ones.

BLEU understates agglutinative languages. English→Malayalam *passes* the target at 0.863 while scoring only 5.2 BLEU, because its teacher scores just 6.1. Malayalam's agglutinative morphology packs into single orthographic words what English expresses in several, so n-gram overlap is structurally low even when the translation is correct. This is exactly why the project gates on the *ratio* rather than absolute BLEU, and why COMET scoring is recommended for Malayalam and Tamil: several near-misses may already pass under a metric that is not overlap-based.

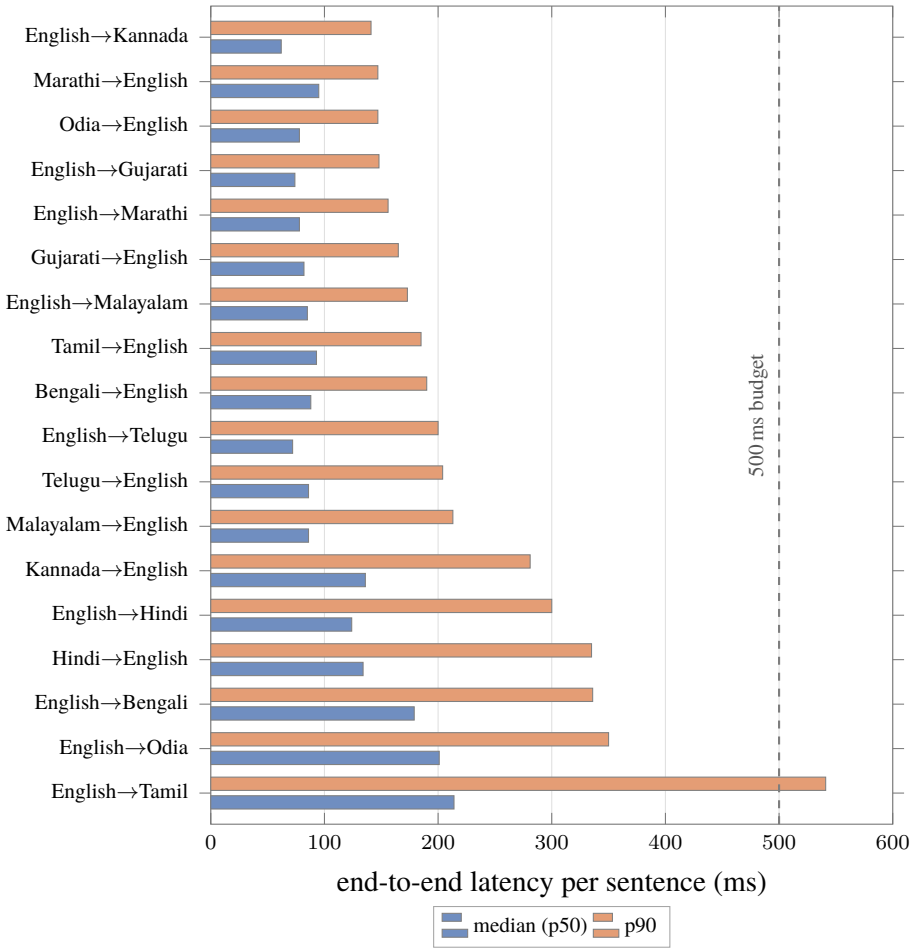


Figure 8.5: Median and p90 end-to-end latency for the 18 deployed directions (INT4 ONNX on the VM host CPU), ordered by p90. Every median sits well inside the 500 ms budget; only English→Tamil crosses it at p90

8.5.3 Deployment Metrics: Size and Latency

Every model compresses by a factor of 4.0, from 410.9 MB to 103.9 MB, which is 48 % below the 200 MB budget. Median latency is comfortably inside the 500 ms budget for every direction. The single exception is English→Tamil, whose p90 of 541 ms exceeds the budget; its median of 214 ms does not. This was measured on a shared VM host under contention, and Tamil's longer output sequences make it the most sensitive direction to host load. It is recorded as a near-miss rather than discounted.

8.5.4 Definition-of-Done Scorecard

Table 8.5: Final scorecard against the four project targets

Target	Metric	Result	Threshold	Status
Quality	Student / teacher BLEU ratio	14 of 22 directions; mean 0.838, median 0.855, best 0.955	≥ 0.80	Partial
Latency	Milliseconds per sentence, quantised	Mean 105–202 ms; p90 141–541 ms	< 500 ms	**Pass**
Size	Quantised artifact, INT4	103.9 MB, every direction	≤ 200 MB	**Pass**
Offline	Inference with networking disabled	Succeeds for every direction	Zero network calls	**Pass**

Three of the four targets pass without exception across all 22 directions. Quality is met on 14 of 22. On the primary development pair, Hindi→English, all four targets are met and the definition of done is complete.

8.6 Qualitative Analysis of Translation Quality

Metrics measure aggregate overlap; they do not show whether output is usable. This section examines what the models actually produce.